

Pengolahan Citra Digital

Bab 2: Dasar-Dasar Pembentukan Citra

Materi ini membahas proses pembentukan citra mulai dari aspek fisik cahaya hingga representasi digitalnya.

0 Contents

1	Persepsi Visual Manusia	2
1.1	Kenapa Perlu Belajar Tentang Mata?	2
1.2	Adaptasi dan Diskriminasi Kecerahan	2
1.3	Fenomena Visual Penting	2
1.3.1	Mach Bands	2
1.3.2	Simultaneous Contrast	2
2	Cahaya dan Spektrum Elektromagnetik	2
2.1	Citra sebagai Energi Elektromagnetik	3
2.2	Sifat Gelombang	3
2.3	Sifat Partikel (Foton)	3
3	Akuisisi Citra dan Model Pembentukan Citra	3
3.1	Bagaimana Citra Terbentuk?	3
3.2	Model Matematis: Fungsi Citra	3
4	Sampling dan Kuantisasi	4
4.1	Dari Dunia Nyata ke Komputer	4
4.2	Representasi Matriks	4
4.3	Kebutuhan Penyimpanan	4
4.4	Contoh Kode Python	5
5	Hubungan Antar Pixel	5
5.1	Konsep Ketetanggaan (<i>Neighborhood</i>)	5
5.1.1	4-Tetangga: $N_4(p)$	6
5.1.2	8-Tetangga: $N_8(p)$	6
5.2	Jenis Ketertetanggaan (<i>Adjacency</i>)	6
5.3	Ukuran Jarak	6
5.4	Contoh Kode Python	6

6 Perangkat Matematika Utama	7
6.1 Operasi Elementwise	7
6.2 Linearitas dan Prinsip Superposisi	8
6.3 Transformasi Affine (Geometris)	8
6.4 Contoh Kode Python	8
Ringkasan	10

1 Persepsi Visual Manusia

1.1 Kenapa Perlu Belajar Tentang Mata?

Mungkin kamu bertanya-tanya, “*Ini kan kuliah teknik, ngapain bahas mata manusia?*” Alasannya cukup masuk akal: sebagian besar citra digital yang kita proses pada akhirnya dikonsumsi oleh manusia. Jadi kalau kita tidak paham cara kerja mata, bisa jadi algoritma yang kita buat terlalu berlebihan atau malah kurang pas.

Bayangkan kamu sibuk *sharpening* (mempertajam) gambar selama berjam-jam, padahal mata manusia sudah punya mekanisme penajaman alaminya sendiri.

1.2 Adaptasi dan Diskriminasi Kecerahan

Mata manusia itu fleksibel banget. Berbeda dengan kamera yang punya rentang dinamis (*dynamic range*) tetap, mata kita terus-menerus menyesuaikan diri dengan tingkat cahaya rata-rata di sekitar kita, yang biasa disebut **level adaptasi** B_a .

- **Rentang total:** Sangat luas – dari cahaya bintang yang redup sampai sinar matahari terik.
- **Rentang pada satu waktu:** Hanya sebagian kecil di sekitar B_a . Makanya kita susah melihat detail di area yang sangat gelap *dan* sangat terang sekaligus.

1.3 Fenomena Visual Penting

1.3.1 Mach Bands

Kalau ada dua area dengan intensitas berbeda yang bersebelahan, mata kita akan “merasakan” garis terang atau gelap *semu* di perbatasannya – padahal secara fisik intensitasnya konstan. Fenomena ini disebut **Mach Bands**.

Implikasi praktis: Karena mata sudah “mempertajam” perbatasan secara alami, kita perlu hati-hati saat menerapkan algoritma *sharpening* berlebihan. Hasilnya bisa terlihat tidak natural.

1.3.2 Simultaneous Contrast

Persepsi kecerahan suatu objek sangat dipengaruhi oleh latar belakangnya. Kotak abu-abu yang sama persis akan terlihat *lebih gelap* jika ditempatkan di atas latar putih, dibandingkan jika ditempatkan di atas latar hitam.

Pertanyaan Reflektif: Jika mata sudah melakukan “penajaman alami” seperti Mach Bands, apakah kita perlu mempertajam citra digital secara agresif lewat algoritma? Tidak selalu – tergantung konteks dan tujuan penggunaannya.

2 Cahaya dan Spektrum Elektromagnetik

2.1 Citra sebagai Energi Elektromagnetik

Citra yang kita olah umumnya berasal dari energi elektromagnetik (EM) – bisa cahaya tampak, inframerah, sinar-X, dan lain-lain. Pemahaman tentang sifat EM penting agar kita tahu *apa yang sebenarnya kita ukur*.

2.2 Sifat Gelombang

Hubungan fundamental antara panjang gelombang λ , frekuensi ν , dan kecepatan cahaya c :

$$\lambda\nu = c \quad (1)$$

di mana $c \approx 3 \times 10^8$ m/s. Cahaya tampak oleh manusia berada pada rentang $\lambda \approx 400\text{--}700$ nm.

2.3 Sifat Partikel (Foton)

Dalam model kuantum, energi satu foton E berbanding lurus dengan frekuensinya:

$$E = h\nu \quad (2)$$

di mana $h = 6.626 \times 10^{-34}$ J · s adalah konstanta Planck.

Intuisi: Sinar-X punya frekuensi jauh lebih tinggi dari cahaya tampak, sehingga energi fotonnya lebih besar. Inilah kenapa sinar-X bisa menembus jaringan tubuh, sementara cahaya tampak tidak bisa.

3 Akuisisi Citra dan Model Pembentukan Citra

3.1 Bagaimana Citra Terbentuk?

Secara sederhana, proses pembentukan citra adalah:

1. Sumber cahaya memancarkan energi ke pemandangan (*scene*).
2. Objek memantulkan atau meneruskan cahaya tersebut.
3. Sensor (misalnya CCD pada kamera digital) menangkap energi yang sampai.
4. Hasilnya direkam sebagai citra digital.

3.2 Model Matematis: Fungsi Citra

Sebuah citra didefinisikan sebagai fungsi dua dimensi:

$$f(x, y) = i(x, y) \cdot r(x, y) \quad (3)$$

di mana:

- $i(x, y)$ – **Illuminasi** (*illumination*): jumlah cahaya yang jatuh pada titik (x, y) . Nilainya selalu positif:

$$0 < i(x, y) < \infty$$

- $r(x, y)$ – **Reflektansi** (*reflectance*): proporsi cahaya yang dipantulkan oleh objek. Nilainya antara 0 dan 1:

$$0 \leq r(x, y) \leq 1$$

Nilai $r = 0$ berarti objek menyerap semua cahaya (hitam sempurna), sedangkan $r = 1$ berarti memantulkan semua cahaya (putih sempurna).

Analogi gampang: Bayangkan kamu menyorotkan senter ke dinding. $i(x, y)$ adalah seberapa kuat senternya, sedangkan $r(x, y)$ adalah seberapa “putih” dindingnya. Hasil yang kamera tangkap adalah perkalian keduanya – $f(x, y)$.

4 Sampling dan Kuantisasi

4.1 Dari Dunia Nyata ke Komputer

Dunia nyata itu kontinu – intensitas cahaya bisa bernilai apa saja di sembarang titik. Komputer, di sisi lain, hanya bisa menangani bilangan diskret. Di sinilah peran *sampling* dan *kuantisasi*.

- **Sampling** – Digitalisasi koordinat spasial (x, y) . Menentukan berapa banyak pixel yang ada (resolusi spasial).
- **Kuantisasi** – Digitalisasi nilai intensitas f . Menentukan berapa banyak tingkat keabuan yang tersedia (resolusi intensitas).

4.2 Representasi Matriks

Sebuah citra berukuran $M \times N$ direpresentasikan sebagai matriks:

$$\mathbf{A} = \begin{pmatrix} a_{0,0} & a_{0,1} & \cdots & a_{0,N-1} \\ a_{1,0} & a_{1,1} & \cdots & a_{1,N-1} \\ \vdots & \vdots & \ddots & \vdots \\ a_{M-1,0} & a_{M-1,1} & \cdots & a_{M-1,N-1} \end{pmatrix} \quad (4)$$

Setiap elemen $a_{i,j}$ disebut **pixel** (*picture element*).

4.3 Kebutuhan Penyimpanan

Jika citra memiliki $L = 2^k$ tingkat intensitas, maka jumlah bit yang dibutuhkan untuk menyimpan citra $M \times N$ adalah:

$$b = M \times N \times k \quad (5)$$

Contoh: Citra *grayscale* 8-bit ($k = 8$, artinya $L = 256$ level) berukuran 1920×1080 membutuhkan:

$$b = 1920 \times 1080 \times 8 = 16,588,800 \text{ bit} \approx 1.98 \text{ MB}$$

Ini adalah ukuran sebelum kompresi. Format JPEG atau PNG mengompresi lebih lanjut.

4.4 Contoh Kode Python

```

1 import numpy as np
2 from PIL import Image
3 import matplotlib.pyplot as plt
4
5 # Baca citra dan konversi ke grayscale
6 img = Image.open("contoh.jpg").convert("L") # "L" = grayscale
7 A = np.array(img) # Konversi ke NumPy
8 array
9
10 M, N = A.shape
11 k = 8 # Bit-depth (asumsi 8-bit)
12 L = 2**k # Jumlah level: 256
13
14 print(f"Ukuran citra : {M} x {N} pixel")
15 print(f"Bit-depth : {k} bit => L = {L} tingkat keabuan")
16 print(f"Kebutuhan bit : {M*N*k:,} bit ({M*N*k/8/1024:.1f} KB)")
17 print(f"Nilai min : {A.min()}")
18 print(f"Nilai max : {A.max()}")
19
20 # Tampilkan citra dan histogram intensitas
21 fig, axes = plt.subplots(1, 2, figsize=(10, 4))
22
23 axes[0].imshow(A, cmap="gray", vmin=0, vmax=255)
24 axes[0].set_title("Citra Grayscale")
25 axes[0].axis("off")
26
27 axes[1].hist(A.ravel(), bins=256, range=(0, 255),
28 color="steelblue", edgecolor="none")
29 axes[1].set_title("Histogram Intensitas")
30 axes[1].set_xlabel("Nilai Pixel")
31 axes[1].set_ylabel("Frekuensi")
32
33 plt.tight_layout()
34 plt.show()

```

Listing 1: Membaca citra dan memeriksa properti dasarnya

5 Hubungan Antar Pixel

5.1 Konsep Ketetanggaan (*Neighborhood*)

Untuk berbagai operasi seperti segmentasi atau deteksi tepi, kita perlu tahu pixel mana yang dianggap “bertetangga” dengan pixel p di koordinat (x, y) .

5.1.1 4-Tetangga: $N_4(p)$

Hanya pixel di atas, bawah, kiri, dan kanan:

$$N_4(p) = \{(x+1, y), (x-1, y), (x, y+1), (x, y-1)\} \quad (6)$$

5.1.2 8-Tetangga: $N_8(p)$

$N_4(p)$ ditambah keempat tetangga diagonal $N_D(p)$:

$$N_8(p) = N_4(p) \cup N_D(p) \quad (7)$$

5.2 Jenis Ketertetanggaan (Adjacency)

- **4-adjacency:** Dua pixel bertetangga jika salah satunya ada di N_4 pixel lainnya.
- **8-adjacency:** Dua pixel bertetangga jika salah satunya ada di N_8 pixel lainnya.
- **m-adjacency (mixed):** Kombinasi 4- dan 8-adjacency untuk menghilangkan ambiguitas jalur. Berguna saat definisi “path” perlu unik.

5.3 Ukuran Jarak

Diberikan dua pixel $p = (x, y)$ dan $q = (u, v)$:

$$D_e(p, q) = \sqrt{(x-u)^2 + (y-v)^2} \quad (\text{Euclidean}) \quad (8)$$

$$D_4(p, q) = |x-u| + |y-v| \quad (\text{City-block / Manhattan}) \quad (9)$$

$$D_8(p, q) = \max(|x-u|, |y-v|) \quad (\text{Chessboard}) \quad (10)$$

Analogi:

- D_e – jarak garis lurus seperti burung terbang.
- D_4 – jarak berjalan di kota Manhattan yang jalannya berbentuk grid.
- D_8 – jarak yang ditempuh raja di papan catur.

5.4 Contoh Kode Python

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def euclidean(p, q):
5     """Jarak Euclidean -- garis lurus."""
6     return np.sqrt((p[0]-q[0])**2 + (p[1]-q[1])**2)
7
8 def city_block(p, q):
9     """City-block (D4) -- berjalan di grid Manhattan."""
10    return abs(p[0]-q[0]) + abs(p[1]-q[1])
11
12 def chessboard(p, q):
```

```

13     """Chessboard (D8) -- gerakan raja di catur."""
14     return max(abs(p[0]-q[0]), abs(p[1]-q[1]))
15
16 # Uji coba
17 p = (0, 0)
18 q = (3, 4)
19 print(f"Pixel p = {p}, pixel q = {q}")
20 print(f"D_euclidean = {euclidean(p, q):.4f}")      # => 5.0000
21 print(f"D_4          = {city_block(p, q)}")        # => 7
22 print(f"D_8          = {chessboard(p, q)}")        # => 4
23
24 # Visualisasi area dengan jarak <= 2 dari pusat
25 center = (5, 5)
26 radius = 2
27 size = 11
28
29 fig, axes = plt.subplots(1, 2, figsize=(10, 5))
30 titles = ["D4 (City-block) <= 2", "D8 (Chessboard) <= 2"]
31
32 for ax, dist_fn, title in zip(axes, [city_block, chessboard], titles):
33     :
34     grid = np.zeros((size, size))
35     for r in range(size):
36         for c in range(size):
37             if dist_fn(center, (r, c)) <= radius:
38                 grid[r, c] = 1
39     ax.imshow(grid, cmap="Blues", vmin=0, vmax=1.5)
40     ax.plot(center[1], center[0], "ro", markersize=8, label="Pusat")
41     ax.set_title(title)
42     ax.legend()
43
44 plt.tight_layout()
45 plt.show()

```

Listing 2: Menghitung tiga jenis jarak antar pixel

6 Perangkat Matematika Utama

6.1 Operasi Elementwise

Hampir semua operasi pada citra bersifat *elementwise* – dilakukan pixel demi pixel secara independen. Misalnya, menjumlahkan dua citra:

$$g(x, y) = f_1(x, y) + f_2(x, y) \quad \forall (x, y) \quad (11)$$

Di Python dengan NumPy, operasi ini dilakukan secara otomatis dan efisien lewat mekanisme *broadcasting*.

6.2 Linearitas dan Prinsip Superposisi

Sebuah operator H disebut **linear** jika memenuhi prinsip superposisi:

$$H[a f_1 + b f_2] = a H[f_1] + b H[f_2] \quad (12)$$

Ini penting karena banyak teori pengolahan citra – termasuk Transformasi Fourier – mengasumsikan sistem yang linear. Kalau operator tidak linear, banyak teori tersebut tidak bisa dipakai langsung.

Cek linearitas dengan cepat: Uji apakah $H[0] = 0$. Jika tidak, operator tersebut pasti tidak linear. Contoh: $H[f] = f + 5$ tidak linear karena $H[0] = 5 \neq 0$.

6.3 Transformasi Affine (Geometris)

Untuk memanipulasi posisi pixel (rotasi, scaling, translasi), digunakan **koordinat homogen**. Transformasi dinyatakan sebagai perkalian matriks 3×3 :

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \mathbf{A} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \quad (13)$$

Beberapa contoh matriks transformasi $\mathbf{A}$:

$$\text{Translasi: } \mathbf{A}_T = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix} \quad (14)$$

$$\text{Rotasi sebesar } \theta : \mathbf{A}_R = \begin{pmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad (15)$$

$$\text{Scaling: } \mathbf{A}_S = \begin{pmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad (16)$$

Keuntungan koordinat homogen: komposisi transformasi cukup dengan mengalikan matriks-matriks tersebut:

$$\mathbf{A}_{\text{total}} = \mathbf{A}_T \cdot \mathbf{A}_R \cdot \mathbf{A}_S$$

6.4 Contoh Kode Python

```
1 import numpy as np
2 import cv2
3 import matplotlib.pyplot as plt
4
5 img = cv2.imread("contoh.jpg", cv2.IMREAD_GRAYSCALE)
6 M, N = img.shape           # Tinggi, Lebar
7
8 # 1. Rotasi 30 derajat terhadap pusat citra
9 pusat = (N // 2, M // 2)
10 A_rot = cv2.getRotationMatrix2D(pusat, angle=30, scale=1.0)
11 img_rot = cv2.warpAffine(img, A_rot, (N, M))
```

```

12
13 # 2. Translasi: geser 50px ke kanan, 30px ke bawah
14 A_trans = np.float32([[1, 0, 50],
15                      [0, 1, 30]])
16 img_trans = cv2.warpAffine(img, A_trans, (N, M))
17
18 # 3. Scaling: perbesar 1.5x
19 A_scale = np.float32([[1.5, 0, 0],
20                      [0, 1.5, 0]])
21 img_scale = cv2.warpAffine(img, A_scale, (N, M))
22
23 # Tampilkan hasil
24 fig, axes = plt.subplots(1, 4, figsize=(14, 4))
25 for ax, im, title in zip(axes,
26                          [img, img_rot, img_trans, img_scale],
27                          ["Original", "Rotasi 30 deg",
28                          "Translasi", "Scaling 1.5x"]):
29     ax.imshow(im, cmap="gray")
30     ax.set_title(title)
31     ax.axis("off")
32 plt.tight_layout()
33 plt.show()
34
35 # Bonus: komposisi transformasi via perkalian matriks 3x3
36 theta = np.radians(30)
37 A_rot3 = np.array([[ np.cos(theta), -np.sin(theta), 0],
38                   [ np.sin(theta),  np.cos(theta), 0],
39                   [ 0,                0,          1]])
40 A_tra3 = np.array([[1, 0, 50],
41                   [0, 1, 30],
42                   [0, 0, 1]])
43
44 # Rotasi LALU translasi -- cukup kalikan matriksnya
45 A_total = A_tra3 @ A_rot3
46 print("Matriks transformasi total (rotasi -> translasi):")
47 print(np.round(A_total, 4))

```

Listing 3: Transformasi Affine: rotasi, translasi, dan scaling

6 Ringkasan

Topik	Konsep Kunci	Persamaan Utama
Persepsi Visual	Adaptasi B_a , Mach Bands, Contrast	–
Spektrum EM	Gelombang & foton	$\lambda\nu = c, E = h\nu$
Model Citra	Iluminasi $\times$ Reflektansi	$f = i \cdot r$
Sampling & Kuant.	Resolusi spasial & intensitas	$b = M \times N \times k$
Ketertetangaan	N_4, N_8, D_e, D_4, D_8	Pers. (8)–(10)
Transformasi	Linearitas, Affine, Koordinat Homogen	Pers. (12), (13)